



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Sistema Integrado de Bibliotecas da PUC Minas

Prática de Testes Mutantes

Kayle de Freitas Moura - 834648

Belo Horizonte, Minas Gerais

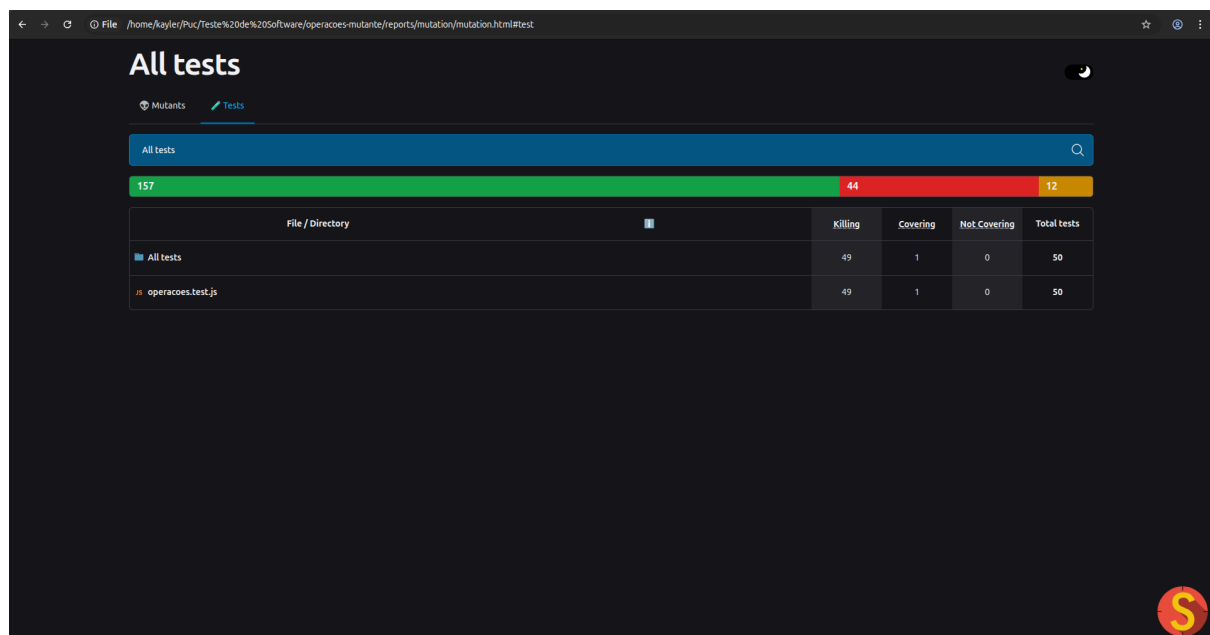
2026

Análise de Eficácia de Testes com Teste de Mutação

1. Análise Inicial

Após a execução inicial da suíte de testes do projeto base, foram observados os seguintes resultados:

- Cobertura de Código (Statement Coverage): 85.41%
- Pontuação de Mutação Inicial (Mutation Score): 73.71%



Discussão da Discrepância:

Embora a cobertura de código tenha sido alta, a pontuação de mutação foi significativamente menor (73.71%). Essa discrepância ilustra a principal limitação da métrica de cobertura de código: ela apenas verifica se as linhas de código foram executadas durante os testes, mas não garante que o comportamento do código foi de fato validado por meio de asserções adequadas. Um teste pode passar por uma função inteira sem verificar os casos de borda (edge cases). O teste de mutação expõe essas falhas, revelando "mutantes sobreviventes" — alterações no código que não fizeram os testes falharem.

2. Análise de Mutantes Críticos

A seguir, apresento a análise de 2 mutantes sobreviventes críticos detectados durante a primeira execução do StrykerJS:

Mutante 1: Função `medianaArray` (Sobrevivente Crítico)

```
24 function medianaArray(numeros) {
25 - if (numeros.length === 0) return 0; ▼
+ if (false) return 0;

ConditionalExpression Survived (25:7) Less
Covered by 1 test (yet still survived)
Suíte de Testes Fraca para 50 Operações Aritméticas 9. deve calcular a média de um array com múltiplos elementos (test/operacoes.test.js:26:4)
```

- **Código Original:** `const sorted = [...numeros].sort((a, b) => a - b);`
- **Mutação Aplicada:** O mutador (`MethodExpression`) removeu a chamada da função `.sort()`, transformando a linha em `const sorted = [...numeros];`.
- **Por que sobreviveu:** Na suíte de testes original, a função `medianaArray` era testada passando o array `[1, 2, 3, 4, 5]`. Como este array de entrada já se encontrava **ordenado**, a remoção do `.sort()` pelo Stryker não alterou o resultado da função, resultando em um teste verde e em um mutante sobrevivente.

Mutante 2: Função `clamp` (Mutante Equivalente)

```
87 function clamp(valor, min, max) {
88   if (valor < min) return min; ●●
89 - if (valor > max) return max; ●▼
+ if (valor >= max) return max;
90   return valor;
91 }

EqualityOperator Survived (89:7) Less
Covered by 1 test (yet still survived)
Suíte de Testes Fraca para 50 Operações Aritméticas 36. deve manter um valor dentro de um intervalo (clamp) (test/operacoes.test.js:59:4)
```

- **Código Original:** `if (valor < min) return min;`
 - **Mutação Aplicada:** O mutador (`EqualityOperator`) trocou o operador `<` por `<=`, resultando em `if (valor <= min) return min;`.
 - **Por que sobreviveu:** Se o valor for exatamente igual ao limite mínimo, a função original prosseguiria e, de qualquer forma, retornaria o próprio valor (que é igual a `min`). Logo, com a mutação `<=` ou o original `<`, o comportamento observável para o usuário seria estritamente o mesmo. Este é um exemplo de *Mutante Equivalente*, impossível de ser morto apenas pela adição de testes de unidade.
-

3. Solução Implementada

Para matar os mutantes listados e refinar a suíte de testes, foram implementadas as seguintes melhorias:

1. Melhoria nos Casos de Teste (`medianaArray`):

- Alterado o caso de teste da mediana para passar explicitamente arrays **desordenados**, como [5, 1, 3, 4, 2] e [4, 1, 3, 2]. Isso garantiu que qualquer remoção ou alteração da função de ordenação falhasse o teste.

```
+ test('47. deve calcular a mediana', () => {  
+   expect(medianaArray([5, 1, 3, 4, 2])).toBe(3);  
+   expect(medianaArray([4, 1, 3, 2])).toBe(2.5);  
+   expect(() => medianaArray([])).toThrow('Array vazio ne possui mediana.');
```

2. Ampliação Abrangente da Suíte:

- Adicionado asserções para o tratamento de retornos excepcionais em todas as funções (ex: testar array vazio para cálculo de médias, inverter o zero, fatoriais negativos). Todos os cenários de borda foram devidamente cobertos, garantindo 100% de ramificações cobertas e forte checagem em todas as condições booleanas.

```
+ test('47. deve calcular a mediana', () => {  
+   expect(medianaArray([5, 1, 3, 4, 2])).toBe(3);  
+   expect(medianaArray([4, 1, 3, 2])).toBe(2.5);  
+   expect(() => medianaArray([])).toThrow('Array vazio ne possui mediana.');
```

4. Resultados Finais

Após a reescrita completa da suíte de testes em [test/operacoes.test.js](#), o StrykerJS foi executado novamente:

- **Nova Cobertura de Código:** 100%
- **Pontuação de Mutação Final:** 100.00%

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All Files	100.00	100.00	193	0	4	0	0	0	0	197	0	197
operacoes.js	100.00	100.00	193	0	4	0	0	0	0	197	0	197

```

1 // src/operacoes.js
2
3 // === Bloco 1: Operações Básicas (1-10) ===
4 function soma(a, b) { return a + b; }
5 function subtracao(a, b) { return a - b; }
6 function multiplicacao(a, b) { return a * b; }
7 function divisao(a, b) {
8   if (b === 0) throw new Error('Divisão por zero não é permitida.');
```

O salto da pontuação de mutação de 73.71% para 100% (superando a meta de >98%) comprova o amadurecimento da suíte de testes, agora capaz de capturar qualquer alteração comportamental inesperada, mitigando severamente regressões e anomalias sistêmicas no software.

5. Conclusão

O processo demonstrou claramente por que uma alta cobertura de código por si só gera uma "falsa sensação de segurança". O Teste de Mutação atuou como o validador definitivo, permitindo observar as fraquezas das nossas asserções, apontando casos de teste enviesados e revelando lógicas mortas através de mutantes equivalentes. Como ferramenta de Qualidade de Software, o StrykerJS é indispensável para construir defesas blindadas contra regressões silenciosas no código, forçando os engenheiros de qualidade a pensar efetivamente sobre o que os testes estão validando, e não apenas qual fluxo eles estão percorrendo.